

**FACULDADE MAURÍCIO DE NASSAU
SISTEMAS DE INFORMAÇÃO — TURMA 5MC**

MARIA EDUARDA FERREIRA MELO

**DOCKER E CI/CD: CONTAINERIZAÇÃO E AUTOMAÇÃO DE
ENTREGA DE SOFTWARE
COM PIPELINES E GITHUB ACTIONS**

**RECIFE
2026**

MARIA EDUARDA FERREIRA MELO

**DOCKER E CI/CD: CONTAINERIZAÇÃO E AUTOMAÇÃO DE
ENTREGA DE SOFTWARE
COM PIPELINES E GITHUB ACTIONS**

Trabalho apresentado à FACULDADE
MAURÍCIO DE NASSAU, como requisito
para avaliação na disciplina de
Tecnologia/Engenharia de Software do
curso de SISTEMAS DE INFORMAÇÃO
(Turma 5MC).

**RECIFE
2026**

RESUMO

A modernização do desenvolvimento de software elevou a relevância de práticas que aumentem previsibilidade, repetibilidade e qualidade nas entregas. Nesse cenário, a containerização com Docker e a adoção de práticas de Integração Contínua e Entrega Contínua (CI/CD) têm papel central. Este trabalho apresenta os fundamentos do Docker, detalhando sua arquitetura baseada em imagens e containers, bem como o processo de construção de imagens por meio de Dockerfiles. São discutidas boas práticas de segurança, performance e reprodutibilidade, incluindo uso de imagens base mínimas, builds em múltiplos estágios, organização de camadas e gerenciamento correto de segredos. Aborda-se o Docker Compose como solução para orquestração local de múltiplos serviços e o Docker Registry como componente essencial para armazenamento, versionamento e distribuição de imagens. Em seguida, são apresentados conceitos de CI/CD e pipelines, destacando etapas típicas, rastreabilidade de artefatos e controles de qualidade. Por fim, descreve-se o GitHub Actions como plataforma de automação integrada ao repositório, incluindo estrutura de workflows e recomendações de segurança. Conclui-se que Docker e CI/CD, quando aplicados com boas práticas, reduzem discrepâncias entre ambientes, aumentam a confiabilidade e apoiam a entrega contínua de software.

ABSTRACT

Software development modernization has increased the importance of practices that improve predictability, repeatability, and delivery quality. In this context, containerization with Docker and the adoption of Continuous Integration and Continuous Delivery (CI/CD) are central. This paper presents Docker fundamentals, detailing its architecture based on images and containers, as well as image building through Dockerfiles. It discusses best practices for security, performance, and reproducibility, including minimal base images, multi-stage builds, layer organization, and proper secrets management. Docker Compose is covered as a solution for local orchestration of multiple services, and Docker Registry is presented as a key component for image storage, versioning, and distribution. Next, CI/CD and pipeline concepts are introduced, highlighting typical stages, artifact traceability, and quality controls. Finally, GitHub Actions is described as a repository-integrated automation platform, including workflow structure and security recommendations. The conclusion is that Docker and CI/CD, when applied with best practices, reduce environment drift, increase reliability, and support continuous software delivery.

SUMÁRIO

1 INTRODUÇÃO

2 DOCKER: FUNDAMENTOS E ARQUITETURA

2.1 Conceitos essenciais

2.2 Containers versus máquinas virtuais

2.3 Componentes e funcionamento (Engine, imagens e camadas)

2.4 Armazenamento, redes e volumes

3 DOCKERFILE E BOAS PRÁTICAS

3.1 Estrutura e instruções principais

3.2 Reprodutibilidade e eficiência de build (cache e camadas)

3.3 Segurança: usuário não-root, superfície de ataque e segredos

3.4 Multi-stage builds e redução de tamanho

4 DOCKER COMPOSE

4.1 Objetivo e cenários de uso

4.2 Redes, volumes e variáveis de ambiente

5 DOCKER REGISTRY

5.1 Conceito, fluxo push/pull e distribuição

5.2 Tags, versionamento e rastreabilidade

5.3 Vulnerabilidades, políticas e governança

6 CI/CD E PIPELINES

6.1 CI, Continuous Delivery e Continuous Deployment

6.2 Etapas típicas de pipeline e critérios de qualidade

6.3 Artefatos, ambientes e estratégia de branches

7 GITHUB ACTIONS

7.1 Conceitos: workflows, jobs, steps e runners

7.2 Exemplo de workflow (build, testes e push de imagem)

7.3 Boas práticas e segurança no Actions

8 CONCLUSÃO

REFERÊNCIAS

1 INTRODUÇÃO

A evolução das práticas de engenharia de software, impulsionada por métodos ágeis e pela necessidade de lançamentos frequentes, aumentou a complexidade operacional dos processos de entrega. Mesmo em projetos pequenos, a diversidade de ambientes (desenvolvimento, testes, homologação e produção) e a quantidade de dependências técnicas (bibliotecas, bancos de dados, caches, filas e serviços externos) elevam o risco de inconsistências. Um dos sintomas mais comuns desse cenário é a divergência entre ambientes, frequentemente resumida pela expressão “funciona na minha máquina”, que revela falhas na padronização do empacotamento e da configuração.

A containerização, especialmente por meio do Docker, tornou-se uma resposta prática para esse problema ao permitir que aplicações sejam empacotadas com suas dependências e configurações de forma consistente. Além disso, a automação de processos por meio de Integração Contínua e Entrega Contínua (CI/CD) reduz erros manuais, acelera feedback e aumenta a confiabilidade. Ao combinar Docker com pipelines automatizados, o time passa a produzir artefatos rastreáveis (por exemplo, imagens versionadas), promovendo previsibilidade e facilitando auditorias e rollback.

Este trabalho tem como objetivo apresentar uma visão abrangente sobre Docker e CI/CD, com foco em Dockerfile e boas práticas, Docker Compose, Docker Registry, conceitos de pipelines e GitHub Actions. A abordagem é descritiva e aplicada, buscando relacionar conceitos com práticas comuns na indústria e com recomendações de documentação oficial.

2 DOCKER: FUNDAMENTOS E ARQUITETURA

2.1 Conceitos essenciais

Docker é uma plataforma que viabiliza a execução de aplicações em containers, unidades isoladas que compartilham o kernel do sistema operacional do host. Containers encapsulam o sistema de arquivos necessário à aplicação, bibliotecas, dependências e variáveis de configuração, fornecendo consistência entre ambientes (DOCKER, 2026). Entre os conceitos centrais destacam-se: imagem (artefato imutável), container (instância em execução), Docker Engine (componente responsável por build e execução) e registry (armazenamento e distribuição de imagens).

2.2 Containers versus máquinas virtuais

Containers e máquinas virtuais (VMs) fornecem isolamento, porém com estratégias distintas. VMs incluem um sistema operacional completo por instância (guest OS) e dependem de um hypervisor, o que tende a aumentar o consumo de recursos e o tempo de

inicialização. Containers, por sua vez, compartilham o kernel do host e isolam processos e recursos, favorecendo rapidez e eficiência. Em contrapartida, exige-se atenção especial a políticas de segurança, permissões e atualização periódica de imagens base, uma vez que artefatos reutilizados podem disseminar vulnerabilidades se não forem gerenciados adequadamente.

2.3 Componentes e funcionamento (Engine, imagens e camadas)

Imagens Docker são estruturadas em camadas. Cada instrução relevante no Dockerfile geralmente cria uma nova camada, e o mecanismo de cache reaproveita camadas quando entradas permanecem as mesmas. Isso melhora desempenho de builds e distribuição: ao enviar a um registry, transferem-se apenas camadas ainda não presentes. Para engenharia de entrega, é útil diferenciar build-time (construção da imagem) e run-time (execução do container com configuração). Containers devem ser tratados como descartáveis: atualizações e correções devem resultar em novas imagens versionadas e substituição da instância, reduzindo variação e facilitando rastreabilidade.

2.4 Armazenamento, redes e volumes

Containers possuem camada de escrita efêmera; ao remover o container, dados podem ser perdidos. Volumes são o mecanismo recomendado para persistência, especialmente em bancos de dados e componentes stateful. Bind mounts mapeiam diretórios do host e são frequentes em desenvolvimento, porém demandam cuidado com permissões e portabilidade. No aspecto de rede, Docker fornece redes virtuais e, em cenários com Compose, facilita comunicação por nome de serviço via DNS interno, aproximando o ambiente local da realidade de produção.

3 DOCKERFILE E BOAS PRÁTICAS

3.1 Estrutura e instruções principais

O Dockerfile descreve a construção de uma imagem por meio de instruções como FROM (imagem base), WORKDIR (diretório de trabalho), COPY (cópia de arquivos), RUN (comandos de build), ENV (variáveis de ambiente), EXPOSE (documentação de portas) e CMD/ENTRYPOINT (processo principal). Por ser parte do processo de entrega, o Dockerfile deve ser versionado e revisado como código, com validação no pipeline.

3.2 Reprodutibilidade e eficiência de build (cache e camadas)

A eficiência de builds depende do aproveitamento de cache e do controle de contexto de build. Uma prática comum é copiar primeiro arquivos de dependências e instalar pacotes antes de copiar o restante do código, maximizando reuso de camadas. Adicionalmente, o uso de .dockerignore evita inclusão de artefatos locais e reduz tamanho

da imagem, impactando positivamente o tempo de execução de pipelines CI/CD.

3.3 Segurança: usuário não-root, superfície de ataque e segredos

Em segurança, recomenda-se executar a aplicação como usuário não privilegiado, reduzir a superfície de ataque por meio de imagens base mínimas e atualizar bases periodicamente. Segredos não devem ser embutidos na imagem (por exemplo, em instruções ENV ou arquivos copiados), pois podem permanecer no histórico de camadas. O fornecimento de credenciais deve ocorrer em run-time por mecanismos apropriados e controlados.

3.4 Multi-stage builds e redução de tamanho

Multi-stage builds separam etapas de compilação e execução. A etapa de build inclui ferramentas e dependências de compilação; a etapa final contém apenas artefatos necessários ao runtime. O resultado tende a ser uma imagem menor, com downloads mais rápidos e menor exposição a pacotes não utilizados. Esse padrão é especialmente relevante em pipelines, pois reduz tempo total de execução.

3.5 Exemplo de Dockerfile (listagem)

Listagem 1 – Exemplo de Dockerfile com boas práticas (multi-stage)

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=build /app/dist ./dist
USER node
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

4 DOCKER COMPOSE

4.1 Objetivo e cenários de uso

Docker Compose organiza múltiplos containers como uma aplicação composta, principalmente em ambiente local de desenvolvimento e testes. Seu valor está na reprodutibilidade: com um único arquivo declarativo, é possível subir aplicação e dependências (banco, cache, filas) com configurações consistentes, reduzindo o tempo de onboarding e evitando divergências entre máquinas.

4.2 Redes, volumes e variáveis de ambiente

Em Compose, redes permitem comunicação por DNS interno com nomes de serviços; volumes viabilizam persistência de dados, especialmente para bancos de dados. Variáveis de ambiente apoiam configuração externa ao código, porém devem ser usadas com cautela para não expor segredos no repositório. Em projetos reais, recomenda-se separar parâmetros sensíveis e adotar mecanismos de secrets.

4.3 Exemplo de docker-compose.yml (listagem)

Listagem 2 – Exemplo de docker-compose.yml para aplicação + banco

```
services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgres://postgres:postgres@db:5432/app
    depends_on:
      - db

  db:
    image: postgres:16-alpine
    environment:
      - POSTGRES_PASSWORD=postgres
      - POSTGRES_DB=app
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

5 DOCKER REGISTRY

5.1 Conceito, fluxo push/pull e distribuição

Registries armazenam imagens e permitem que times e ambientes consumam o mesmo artefato versionado. O fluxo usual envolve construir a imagem no CI, aplicar tags (versão e/ou hash de commit), publicar no registry (push) e baixar em servidores ou clusters (pull). Essa prática aumenta rastreabilidade e reduz risco de “rebuild” divergente entre ambientes.

5.2 Tags, versionamento e rastreabilidade

O versionamento por tags semânticas (por exemplo, 1.3.0) e tags por commit facilita auditoria e rollback. Embora a tag latest seja conveniente, ela pode reduzir reprodutibilidade se usada como referência única. Labels adicionais podem registrar metadados como origem do build e data de geração.

5.3 Vulnerabilidades, políticas e governança

Registries modernos podem integrar varredura de vulnerabilidades, políticas de retenção (remoção de imagens antigas), controle de acesso e mecanismos de assinatura/verificação. Em governança, o foco é garantir que somente imagens aprovadas, rastreáveis e com controle de origem sejam implantadas em ambientes sensíveis.

6 CI/CD E PIPELINES

6.1 CI, Continuous Delivery e Continuous Deployment

CI/CD organiza automações do desenvolvimento à entrega. Em Integração Contínua (CI), cada mudança dispara build e testes, reduzindo defeitos e integração tardia. Em Continuous Delivery, o sistema permanece sempre pronto para release, com etapas automatizadas e, quando necessário, aprovação humana. Em Continuous Deployment, mudanças que atendem aos critérios seguem automaticamente até produção (HUMBLE; FARLEY, 2010).

6.2 Etapas típicas de pipeline e critérios de qualidade

Uma pipeline típica inclui validações rápidas (lint e análise estática), testes unitários, testes de integração, build de artefato (imagem Docker), publicação no registry e deploy em ambientes de teste/homologação. Critérios de qualidade podem incluir cobertura mínima, aprovação de pull request e auditoria de dependências. A automação aumenta previsibilidade e reduz erro humano, fornecendo logs e evidências para inspeção.

6.3 Artefatos, ambientes e estratégia de branches

Em ambientes maduros, promove-se o mesmo artefato entre ambientes, em vez de reconstruir para produção. Essa prática reduz variação e garante que o que foi testado é o que será implantado. Estratégias de branches variam (trunk-based development ou modelos com branches de release), mas a consistência do pipeline em pull requests e merges é essencial para qualidade.

7 GITHUB ACTIONS

7.1 Conceitos: workflows, jobs, steps e runners

GitHub Actions é uma plataforma de automação integrada ao GitHub, permitindo pipelines acionadas por eventos como push, pull request e tags (GITHUB, 2026). Workflows são definidos em arquivos YAML; jobs agrupam steps executados em runners; e steps podem ser comandos ou actions reutilizáveis. A integração com o repositório facilita rastreabilidade e padronização de validações.

7.2 Exemplo de workflow (build, testes e push de imagem)

Listagem 3 – Exemplo de workflow com build e publicação de imagem

```

name: ci

on:
  push:
    branches: ["main"]
  pull_request:

jobs:
  test-build:
    runs-on: ubuntu-latest
    permissions:
      contents: read

    steps:
      - uses: actions/checkout@v4

      - name: Build image
        run: docker build -t myapp:${{ github.sha }} .

      - name: Run tests (example)
        run: docker run --rm myapp:${{ github.sha }} npm test

  publish:
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    needs: [test-build]
    permissions:
      contents: read
      packages: write

    steps:
      - uses: actions/checkout@v4

      - name: Login to registry
        run: |
          echo "${{ secrets.REGISTRY_TOKEN }}" | \
          docker login registry.example.com \
          -u "${{ secrets.REGISTRY_USER }}" --password-stdin

      - name: Build and push
        run: |
          docker build -t registry.example.com/myapp:${{ github.sha }} .
          docker push registry.example.com/myapp:${{ github.sha }}

```

7.3 Boas práticas e segurança no Actions

Em GitHub Actions, recomenda-se aplicar o princípio do menor privilégio (permissions por job), manter segredos em secrets e evitar impressão em logs, fixar versões de actions para reduzir riscos de cadeia de suprimentos e separar ambientes de deploy com controles (por exemplo, aprovações). O objetivo é manter automação rápida, porém segura e auditável.

8 CONCLUSÃO

Docker e CI/CD oferecem ganhos complementares: Docker padroniza execução e empacotamento, enquanto CI/CD padroniza e automatiza validações, builds e entregas. Dockerfiles bem estruturados, com boas práticas de segurança e reprodutibilidade, produzem imagens menores e confiáveis, impactando diretamente o desempenho de pipelines. Docker Compose facilita ambientes locais consistentes, reduzindo tempo de onboarding e problemas de integração. Registries completam o ciclo ao armazenar e distribuir imagens versionadas, tornando a entrega auditável e permitindo promover o mesmo artefato entre ambientes.

Em CI/CD, pipelines bem definidas reduzem falhas humanas e aumentam rastreabilidade, especialmente quando integradas a ferramentas como GitHub Actions. Contudo, os benefícios dependem de governança: controle de segredos, permissões mínimas, atualização de bases e análise de vulnerabilidades são práticas indispensáveis. Assim, a adoção conjunta de Docker e CI/CD, quando feita com disciplina, melhora qualidade, velocidade e confiabilidade das entregas de software.

REFERÊNCIAS

ABNT. NBR 14724: Informação e documentação — Trabalhos acadêmicos — Apresentação. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2011.

DOCKER. Docker Documentation. [S. l.], 2026. Disponível em: <https://docs.docker.com/>. Acesso em: 5 maio 2026.

GITHUB. GitHub Actions Documentation. [S. l.], 2026. Disponível em: <https://docs.github.com/actions>. Acesso em: 5 maio 2026.

HUMBLE, Jez; FARLEY, David. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston: Addison-Wesley, 2010.

TURNBULL, James. The Docker Book: Containerization is the new virtualization. [S. l.]: James Turnbull, 2014.